

Character Literals programs contd :

```
void main()
{
    char c1 = 'A';
    char c2 = 65;
    char c3 = '\u0041';

    IO.println("c1 = "+c1+", c2 =" +c2+", c3 =" +c3);
}

void main()
{
    int x = 'A';
    int y = '\u0041';
    IO.println("x = "+x+" y =" +y);
}

//Every escape sequence is char literal

void main()
{
    char ch = '\n';
    IO.print("Hello");
    IO.println(ch);

    IO.println("\nHello Java");
    IO.println("\nHello Java\n");
}

void main()
{
    IO.println("Min value :"+(int)Character.MIN_VALUE);
    IO.println("Max value :"+(int)Character.MAX_VALUE);
    IO.println("Size :"+Character.SIZE);

    IO.println(".....");

    int min = '\u0000';
    int max = '\uffff';
    IO.println(min);
    IO.println(max);
}
```

String Literal :

String is a predefined class available in java.lang (Library) Package.

String is a collection of **alpha-numeric character** which is enclosed by **double quotes**. These characters can be alphabets, numbers, symbol or any special character.

Example:
"India"
"Hyderabad"
"B-61, S R Nagar"

In java we can create String object by using following 3 ways:

- 1) By using String Literal
String str1 = "India";
- 2) By using new keyword
String str2 = new String("Hyderabad");
- 3) By using Character array (Char Literal)[Old Technique]
char ch[] = {'R', 'A', 'J'};
String s3 = new String(ch);

//Program :

```
void main(String [] args)
{
    //String Literal [Popular]
    String s1 = "Java";

    //By using new Keyword
    String s2 = new String("India");

    //By using Character literal [Old technique]
    char []c1 = {'R', 'A', 'J'};
    String s3 = new String(c1);

    IO.println(s1);
    IO.println(s2);
    IO.println(s3);
}

//String is collection of alpha-numeric character

void main()
{
    String x = "B-61 Hyderabad";
    IO.println(x);

    String y = "123";
    IO.println(y);

    String z = "67.90";
    IO.println(z);

    String p = "A";
    IO.println(p);
}

//IQ
void main()
{
    String s = 15 + 29 + "Ravi" + 40 + 40;
    IO.println(s);
}

void main()
{
    String s1 = 80 + 90 + "Java" - 12; //error
    IO.println(s1);
}
```

What is a local variable in java ?

* If we declare a variable inside the **method OR block OR constructor body** then It is called local OR temporary OR Automatic OR Stack Variable.

Example :
void accept()
{
int x = 100; //x is a local variable
}

* As far as its accessibility is concerned, It is accessible through method, block OR constructor body, where we have declared this local variable.

* A local variable must be initialized by **developer before use** because a local variable cannot have default values.

* We cannot apply any kind of access modifier on local variable except final.

Example :
public void accept()
{
public int x = 10; //error [Only final is applicable]
}

* A local variable we **cannot** use without pre declaration.

Example :
public void accept()
{
IO.println(x); //error
int x = 100;
}

* All the local variables are executed inside a method, block OR constructor body. All the methods are executed in a very special memory area called Stack Area. [All the methods are executed in Stack Memory area]

* It works on LIFO basis [Last In First Out]

* Every time we call a method in Java, Internally one **new stack frame** will be created inside Stack memory.

Diagram for the Program :

```
void main()
{
    IO.println("Main method started");
    m1();
    IO.println("Main method ended");
}

void m1()
{
    IO.println("M1 method started");
    m2();
    IO.println("M1 method ended");
}

void m2()
{
    IO.println("M2 method started");
    IO.println("M2 method ended");
    int x = 100;
}
```



Punctuators OR Separators :

It is also called separators.

It is used to inform the compiler how things are grouped in the code.

() {} [] ; , . @

Operators :



It is a symbol which describes that how a calculation will be performed on operands.

It performs specific operations on one, two, or three operands, and then return a result.

Types of Operators:

In general, three flavours of operators are available in Java: Unary, binary and ternary.